Project 20 (Real-World Example) — SQLSage-Natural-Language-To-SQL-Analytics-Agent

Type: NL-to-SQL analytics agent · **Stack:** Python, LangChain (structured output), SQLite (stdlib `sqlite3`), OpenAI SDK, Pydantic · **Level:** Intermediate

Tagline: Ask a database a question in plain English — "how many orders are completed?" — and get an answer, with a **safety guard** that runs the generated SQL only if it's a single read-only `SELECT`.

Charter (what this project is)

Business questions live in a database, but the people asking them don't write SQL. **SQLSage** turns a plain-English question into a SQL query, checks that the query is safe to run, executes it against a SQLite database, and phrases the result back in one sentence. The twist that makes it production-sane is the **guard**: an LLM can write anything, so before any query touches the database, a deterministic check rejects everything that isn't a single read-only `SELECT` — no `INSERT`, `UPDATE`, `DELETE`, `DROP`, or chained statements. It's structured output (Doc05) to get the SQL, a plain-Python guard you can fully test with no model, and a grounded answer over the rows.

Jump to: Project Structure · Setup · How testing works · Step 1 · Step 2 · Step 3

The Story — what you're building

A question comes in. SQLSage reads the database **schema** (the `CREATE TABLE` statements), hands it to the model, and asks for a single read-only `SELECT` that answers the question — returned as **structured output**, so you get a clean SQL string, not prose. Before that SQL runs, a deterministic **guard** checks it: it must start with `SELECT`, contain no write or schema keyword, and be a single statement. Only then does the **runner** execute it against SQLite and return the rows, and the **answer** step turns those rows into one plain sentence.

The lesson: letting a model write SQL is powerful and dangerous, and the safety is a **deterministic guard**, not a hope. The guard (SQL string in, safe/unsafe out) and the database runner (query in, rows out) need no model, so you test them with hand-written SQL and a tiny in-memory database. The model only writes the query and phrases the result.

Project Structure (the professional layout)

`main.py` takes a question; all code in `src/`.

```
<repo folder>/ (the SQLSage repo — full name in the heading above)
├── data/                       the SQLite database lives here (built by seed)
├── src/
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py
│   ├── db/
│   │   ├── __init__.py
│   │   ├── seed.py             build_sample_db()        (Step 1)
│   │   ├── schema.py           schema_text()            (Step 1)
│   │   └── runner.py           run_select()             (Step 1)
│   ├── utils/
│   │   ├── __init__.py
│   │   └── logger.py
│   ├── __init__.py
│   ├── guard.py                is_safe_select()         (Step 1)
│   ├── schemas.py              SQLQuery                  (Step 2)
│   ├── sql_agent.py            generate_sql()            (Step 2)
│   ├── answer.py               summarize_answer()        (Step 2)
│   ├── config.py
│   ├── exceptions.py
│   └── pipeline.py             ask()                     (Step 3)
├── tests/
│   ├── integration/test_live_ask.py
│   └── unit/                   test_config, test_guard, test_db
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
├── README.md
└── requirements.txt
```

How the structure grows, step by step

- **Step 1** builds the standard setup, `src/db/` (seed + schema + runner) and `src/guard.py` (the deterministic safety check) with tests — the guard and runner tested with no model.
- **Step 2** adds `src/schemas.py`, `src/sql_agent.py` (structured SQL generation), and `src/answer.py` (the grounded answer).
- **Step 3** adds `src/pipeline.py`, `main.py`, and the integration test.

Setup (do this once, before Step 1)

```
cd SQLSage-Natural-Language-To-SQL-Analytics-Agent
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install langchain-openai openai pydantic python-dotenv
pip freeze > requirements.txt
```

`.env.example` (copy to `.env`): `OPENAI_API_KEY=sk-...`, `LOG_LEVEL=INFO`, `DB_PATH=data/shop.db`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`, `data/*.db`. Every folder under `src/` gets an empty `__init__.py`. SQLite ships with Python — no database server to install. `LICENSE` is the MIT text.

How testing works in this project

Two layers, one style (plain OK/WRONG scripts, like the other projects):

- **`tests/unit/`** — no model: `test_config`; `test_guard` (the safety check — a plain `SELECT` passes; `INSERT`, `DELETE`, `DROP`, and a chained `SELECT ...; DROP ...` are all rejected); `test_db` (build the sample database in a temp file, run a known `SELECT`, check the rows).
- **`tests/integration/test_live_ask.py`** — real model: a plain-English question is turned into SQL, guarded, run, and answered correctly.

Each step ends with a **Tests for this step** box.

Step 1 — The Database and the Safety Guard

What this step does: the sample database (seed + schema reader + query runner) and the deterministic `is_safe_select` guard — the parts that store the data and keep an unsafe query from ever running, with no model.

Why this step matters: the guard is what makes "let a model write SQL" safe to ship; proving it with hand-written SQL means Step 2's model call is the only unproven piece left.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write the standard setup (`config` / `exceptions` / `utils/logger` / `models/llm`). `exceptions.py` gets a second error, `UnsafeQueryError`. 2. Write `src/db/seed.py`: `build_sample_db(db_path)` — create `customers` and `orders` tables and insert a few rows (only if the file doesn't exist yet). `src/db/schema.py`: `schema_text(db_path)` — return the `CREATE TABLE` statements as one string (for the prompt). `src/db/runner.py`: `run_select(db_path, sql)` — execute the query and return the rows. 3. Write `src/guard.py`: `is_safe_select(sql)` — `True` only if the query starts with `SELECT`, has no write/schema keyword, and is a single statement.

Tests for this step: `test_guard` — `SELECT ...` passes; `INSERT` / `DELETE` / `DROP` /chained are rejected. `test_db` — the seeded database returns the expected rows. `test_config`.

Step 2 — The SQL Generator and the Answer

What this step does: `generate_sql(model, schema, question)` — structured output that turns the question into a `SELECT`; and `summarize_answer(model, question, rows)` — phrase the result rows in one plain sentence.

Why this step matters: structured output gives you a clean SQL string (not prose you have to parse), and the answer step turns raw rows into something a person can read.

What's new vs. Step 1: `src/schemas.py`, `src/sql_agent.py`, `src/answer.py`. **What stays the same:** the database and the guard.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write `src/schemas.py`: `SQLQuery(sql)` (a Pydantic model). 2. Write `src/sql_agent.py`: a prompt that gives the schema and asks for a single read-only `SELECT`, and `generate_sql = (prompt | model.with_structured_output(SQLQuery)).invoke(...)` returning `.sql`. 3. Write `src/answer.py`: `summarize_answer(model, question, rows)` — answer in one sentence using only the rows; "no matching data" when the rows are empty.

Tests for this step: proven live in Step 3; for now, `generate_sql` returns a string and `summarize_answer` returns a string given hand-made rows.

Step 3 — The Pipeline (final)

What this step does: `ask` ties it together — read the schema, generate the SQL, **guard it**, run it, and summarize the rows — plus `main.py` and the integration test.

Why this step matters: this is the one call a user makes: a plain-English question in, a checked query and a readable answer out.

What's new vs. Step 2: `src/pipeline.py`, `main.py`. **What stays the same:** everything else.

Stuck? Hint 1 · Hint 2 · Solution

What to do: 1. Write `src/pipeline.py`: `ask(model, db_path, question)` — `schema_text` → `generate_sql` → raise `UnsafeQueryError` if `is_safe_select` is `False` → `run_select` → `summarize_answer`, returning the answer, the SQL, and the rows. 2. Write `main.py` (take a question, build the database if missing, print the SQL and the answer) and `tests/integration/test_live_ask.py`.

Final Deliverable: an agent that answers plain-English questions about a SQLite database, running only guarded read-only queries.

Tests for this step: `test_live_ask` — "how many orders are completed?" is turned into a `SELECT`, guarded, run, and answered with the right count.

Run, Test and Configure the Final Project

```
python main.py "How many orders are completed?"
python -m tests.unit.test_config
python -m tests.unit.test_guard
python -m tests.unit.test_db
python -m tests.integration.test_live_ask
```

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key
`LOG_LEVEL`	no	`INFO`	Log verbosity
`DB_PATH`	no	`data/shop.db`	The SQLite database to query

Checklist Before You Call This Done

- [ ] The SQL comes back as structured output (a clean query string)
- [ ] The guard rejects anything that isn't a single read-only `SELECT`
- [ ] The guard and the runner are deterministic, tested with no model
- [ ] The answer is phrased from the returned rows, not invented
- [ ] An unsafe generated query raises `UnsafeQueryError` and never runs

Publish It to GitHub

Name the repo `SQLSage-Natural-Language-To-SQL-Analytics-Agent`, add the `README.md`, push. "Ask my database in plain English — safely" is a demo every data team wants.

Status

Not started.

Stuck? Ask for **Hint 1** or **Hint 2** before the solution. Build the guard and the database runner (Step 1) first — they're deterministic and need no model.